COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S1
DATE PERFORMED:	Jul 21, 2026	**DATE SUBMITTED:**	Aug 1, 2026
NAME:	Jave Joaquin Obias	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 3 **[Linked Lists]**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Construct C++ code for a singly and doubly linked list in C++

A.1. Singly Linked List

SOURCE CODE

```
#include <iostream>
#include "CPE010_ILOA1_singly.h"

int main() {
   //create the node pointing to null
   SingleList<char> *head = NULL;
   SingleList<char> *second = NULL;
   SingleList<char> *third = NULL;
   SingleList<char> *fourth = NULL;
   SingleList<char> *fifth = NULL;
   SingleList<char> *last = NULL;

   //create the nodes
   head = new SingleList<char>;
   second = new SingleList<char>;
   third = new SingleList<char>;
   fourth = new SingleList<char>;
   fifth = new SingleList<char>;
   last = new SingleList<char>;

   head->data = 'C'; //stores the 'C' in the first node
   head->next = second; //first node pointing to second

   second->data = 'P'; //stores the 'P' in the second node
   second->next = third; //second node pointing to third

   third->data = 'E'; //stores the 'E' in the third node
   third->next = fourth; //third node pointing to fourth

   fourth->data = '0'; //stores the '0' in the fourth node
   fourth->next = fifth; //fourth node pointing to fifth
```

```
   fifth->data = '1'; //stores the '1' in the fifth node
   fifth->next = last; //fifth node pointing to last node

   last->data = '0'; //stores the '0' in the last node
   last->next = nullptr; //last node pointing to null

   return 0;
}

//[C] -> [P] -> [E] -> [0] -> [1] -> [0] -> NULL
```

DISCUSSION

In this program it creates a single linked list that stores one character in each node and links each node to the next node using pointers. The last node points into null that indicates there are no more nodes after it. It only creates and links the nodes but it cannot display the output, and the possible output is "C -> P -> E -> 0 -> 1 -> 0 -> NULL".

Table 3-1. Output of Initial/Simple Implementation

SOURCE CODE (Traversal)

```
void listTraversal(SingleList<T>* head){

   while(head != nullptr){
       std::cout <<head->data;

       //check if the next line is not equal to NULL
       if (head->next != nullptr){
           std::cout << " -> ";
       }

       //move to the next node
       head = head->next;
   }

   //go to the next line after displaying the list
   std::cout<<std::endl;
}
```

OUTPUT

```
C -> P -> E -> 0 -> 1 -> 0
```

SOURCE CODE (Insertion at head)

```
void sllinsertHead(T newdata, SingleList<T>** currentHead){

   //create a new node
```

```
    SingleList<T>* newNode = new SingleList<T>;

    //store the new data in the node
    newNode->data = newdata;

    //point the new node to the current head
    newNode->next = *currentHead;

    //make the new node the new head
    *currentHead = newNode;
}
```

SOURCE CODE (Insertion at any part of the list)

```
void sllGeneralInsertion(T newdata, SingleList<T> * prevNode){

    //check if the previous node is null
    if(prevNode==nullptr){
        std::cout<<"The previous node is null." << std::endl;
        return;
    }

    //Allocate new node
    SingleList<T>* newNode = new SingleList<T>;

    //Store data in the new node
    newNode->data = newdata;

    //Point new node to the node previous node was pointing to
    newNode->next = prevNode->next;

    //points previous node to the new node
    prevNode->next = newNode;
}
```

SOURCE CODE (Insertion at the end)

```
void sllinsertEnd(T newData, SingleList<T>** head){

    //Allocate new node
    SingleList<T>* newNode = new SingleList<T>;

    //Dereference to the head node
    SingleList<T>* currentNode = *head;

    //Store data in new node
    newNode->data = newData;

    //Point next of new node to NULL
    newNode->next = nullptr;

    //check if the list is empty
    if(currentNode == nullptr){
```

```
        *head = newNode;
        return;
    }

    //Traverse the list until next of the node is null
    while(currentNode->next != nullptr){

        //Iteration going to one node to another
        currentNode = currentNode->next;
    }

    //connect the last node to the new node
    currentNode->next = newNode;
}
```

SOURCE CODE (Deletion of a node)

```
void sllDelete(T findData, SingleList<T>** head){

    //check if the list is empty
    if(*head == nullptr) return;

    //create pointers for the current and previous node
    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    //Search for the node:
    while(currNode != nullptr && currNode->data != findData){

        //move the previous node to the current node
        prevNode = currNode;

        //move the current node to the next node
        currNode = currNode->next;
    }

    //if data is not found
    if(currNode == nullptr) return;

    //delete the head node(the head node is the findData)
    if (prevNode == nullptr){
        *head = currNode->next;
    }else{

        //connect the previous node to the next node
        prevNode->next = currNode->next;
    }

    //delete the node
    delete currNode;
}
```

Table 3-2. Code for the List Operations

a.) SOURCE CODE

```
//singly.h
void listTraversal(SingleList<T>* head){

   while(head != nullptr){
       std::cout <<head->data;

       //check if the next line is not equal to NULL
       if (head->next != nullptr){
           std::cout << " -> ";
       }

       //move to the next node
       head = head->next;
   }

   //go to the next line after displaying the list
   std::cout<<std::endl;
}
```

```
//singly.cpp
//Traverse the list by passing the head of the created list into the
function.
std::cout <<"Traverse the original list: " << std::endl;
listTraversal(head);
```

CONSOLE

```
Traverse the original list:
C -> P -> E -> 1 -> 0 -> 1
```

b.) SOURCE CODE

```
//singly.h
void sllinsertHead(T newdata, SingleList<T>** currentHead){

   //create a new node
   SingleList<T>* newNode = new SingleList<T>;

   //store the new data in the node
   newNode->data = newdata;

   //point the new node to the current head
   newNode->next = *currentHead;

   //make the new node the new head
   *currentHead = newNode;
```

```
}
```

```
//singly.cpp
//Insert the element 'G' at the start of the list to replace the current node. Output should now show "GCPE101".
sllinsertHead('G', &head);
std::cout <<"\nInsert G at the start: " << std::endl;
listTraversal(head);
```

CONSOLE

```
Insert G at the start:
G -> C -> P -> E -> 1 -> 0 -> 1
```

c.) SOURCE CODE

```
//singly.h
void sllGeneralInsertion(T newdata, SingleList<T> * prevNode){

   //check if the previous node is null
   if(prevNode==nullptr){
       std::cout<<"The previous node is null." << std::endl;
       return;
   }

   //Allocate new node
   SingleList<T>* newNode = new SingleList<T>;

   //Store data in the new node
   newNode->data = newdata;

   //Point new node to the node previous node was pointing to
   newNode->next = prevNode->next;

   //points previous node to the new node
   prevNode->next = newNode;
}
```

```
//singly.cpp
//Insert an element "E" with the previous node element being "P". Output should now show "GCPEE101".
sllGeneralInsertion('E', head->next->next);
std::cout <<"\nInsert E after P: " << std::endl;
listTraversal(head);
```

CONSOLE

```
Insert E after P:
G -> C -> P -> E -> E -> 1 -> 0 -> 1
```

d.) SOURCE CODE

```
//singly.h
void sllDelete(T findData, SingleList<T>** head){

   //check if the list is empty
   if(*head == nullptr) return;

   //create pointers for the current and previous node
   SingleList<T>* currNode = *head;
   SingleList<T>* prevNode = nullptr;

   //Search for the node:
   while(currNode != nullptr && currNode->data != findData){

       //move the previous node to the current node
       prevNode = currNode;

       //move the current node to the next node
       currNode = currNode->next;
   }

   //if data is not found
   if(currNode == nullptr) return;

   //delete the head node(the head node is the findData)
   if (prevNode == nullptr){
       *head = currNode->next;
   }else{

       //connect the previous node to the next node
       prevNode->next = currNode->next;
   }

   //delete the node
   delete currNode;
}
```

```
//singly.cpp
//Delete the node containing the element C.
sllDelete('C', &head);
std::cout <<"\nDelete C: " << std::endl;
listTraversal(head);
```

CONSOLE

```
Delete C:
G -> P -> E -> E -> 1 -> 0 -> 1
```

e.) SOURCE CODE

```
//singly.h
void sllDelete(T findData, SingleList<T>** head){

   //check if the list is empty
   if(*head == nullptr) return;

   //create pointers for the current and previous node
   SingleList<T>* currNode = *head;
   SingleList<T>* prevNode = nullptr;

   //Search for the node:
   while(currNode != nullptr && currNode->data != findData){

       //move the previous node to the current node
       prevNode = currNode;

       //move the current node to the next node
       currNode = currNode->next;
   }

   //if data is not found
   if(currNode == nullptr) return;

   //delete the head node(the head node is the findData)
   if (prevNode == nullptr){
       *head = currNode->next;
   }else{

       //connect the previous node to the next node
       prevNode->next = currNode->next;
   }

   //delete the node
   delete currNode;
}
```

```
//singly.cpp
//Delete the node containing the element P.
sllDelete('P', &head);
std::cout <<"\nDelete P: " << std::endl;
listTraversal(head);
```

CONSOLE

```
Delete P:
G -> E -> E -> 1 -> 0 -> 1
```

f.) SOURCE CODE

```
//singly.h
void listTraversal(SingleList<T>* head){

   while(head != nullptr){
       std::cout <<head->data;

       //check if the next line is not equal to NULL
       if (head->next != nullptr){
           std::cout << " -> ";
       }

       //move to the next node
       head = head->next;
   }

   //go to the next line after displaying the list
   std::cout<<std::endl;
}
```

```
//singly.cpp
//Show the elements in the list. Output should be "GEE101".
std::cout <<"\nFinal list: " << std::endl;
listTraversal(head);
```

CONSOLE

```
Final list:
G -> E -> E -> 1 -> 0 -> 1
```

Table 3-3. Code and Analysis for Singly Linked Lists

A.2. Doubly Linked List

SOURCE CODE

```
//doubly.cpp
#ifndef Double_LL_H
#define Double_LL_H

#include <iostream>

template <typename T>
class DoubleList {
```

```
    public:
        T data;
        DoubleList<T>* next = nullptr;
        DoubleList<T>* prev = nullptr;
};

//----------------------
// Creating a node
//----------------------
template <typename T>
DoubleList<T>* new_Node(T newData) {
    // dynamically allocate memory for a new node
    DoubleList<T>* newNode = new DoubleList<T>;

    // store the data in the node
    newNode->data = newData;

    // since this is a new node, it does not point to any other node
    newNode->next = nullptr;
    newNode->prev = nullptr;
    return newNode;
}

//----------------------
// Traversal
//----------------------
template <typename T>
void DllTraverse(DoubleList<T>* currentNode) {
    DoubleList<T>* tail = nullptr;

    // 1. Forward Traversal
    std::cout << "Forward Traversal:\n";
    while (currentNode != nullptr) {
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
    std::cout << "\n"; // Newline so Reverse Traversal starts on a new line

    // 2. Reverse Traversal
    std::cout << "Reverse Traversal:\n";
    while (tail != nullptr) {
        std::cout << tail->data << " ";
        tail = tail->prev;
    }
    std::cout << "\n\n"; // Newlines to space out calls from main()
}

//----------------------
// Insertion at head
//----------------------
template <typename T>
void DllInsertHead(T newData, DoubleList<T>*& currenthead) {
```

```
    DoubleList<T>* newNode = new_Node(newData);

    newNode->next = currenthead;
    newNode->prev = nullptr;
    if (currenthead != nullptr) {
        currenthead->prev = newNode;
    }
    currenthead = newNode;
}

// ----------------------
// Insertion at the end
// ----------------------
template <typename T>
void DllInsertEnd(T newData, DoubleList<T>*& currenthead) {
    // create a new node
    DoubleList<T>* newNode = new_Node(newData);

    // If the list is empty, new node becomes the head
    if (currenthead == nullptr) {
        currenthead = newNode;
        return;
    }

    // traverse using a temporary pointer until the last node
    DoubleList<T>* tail = currenthead;
    while (tail->next != nullptr) {
        tail = tail->next;
    }

    // connect the last node to the new node
    tail->next = newNode;

    // connect the prev node to the new node
    newNode->prev = tail;
}

//----------------------
// General Insertion
//----------------------
template <typename T>
void DllgeneralInsert(T newData, DoubleList<T>* prevNode) {
    if (prevNode == nullptr) {
        std::cout << "Previous node cannot be null.\n";
        return;
    }

    DoubleList<T>* newNode = new_Node(newData);

    // Link the new node
    newNode->next = prevNode->next;
    newNode->prev = prevNode;
```

```
    // If there is a node after prevNode, fix its prev pointer
    if (prevNode->next != nullptr)
        prevNode->next->prev = newNode;

    // Finish the insertion
    prevNode->next = newNode;
}

//----------------------
// Deletion
//----------------------

// Deletes a single node by value
template <typename T>
void DllDelete(T findData, DoubleList<T>*& head) {
    if (head == nullptr)
        return;

    DoubleList<T>* currNode = head;

    // Search for the node containing findData
    while (currNode != nullptr && currNode->data != findData)
        currNode = currNode->next;

    // Node not found
    if (currNode == nullptr)
        return;

    // Update previous node's next pointer or move head pointer
    if (currNode->prev != nullptr)
        currNode->prev->next = currNode->next;
    else
        head = currNode->next;

    // Update next node's prev pointer
    if (currNode->next != nullptr)
        currNode->next->prev = currNode->prev;

    delete currNode;
}
// Deletes the entire list to free memory
template <typename T>
void DllDeleteList(DoubleList<T>*& head) {
    DoubleList<T>* currNode = head;

    while (currNode != nullptr) {
        DoubleList<T>* temp = currNode;
        currNode = currNode->next;
        delete temp;
    }
    head = nullptr;
}
```

```
#endif
```

```
//doubly.cpp
#include <iostream>
#include "CPE010_HOA3_ILOA.h"

int main() {
   // 'C' 'P' 'E'

   // head node
   DoubleList<char>* head = new_Node('C');

   // second node
   DoubleList<char>* second = new_Node('P');

   // third node
   DoubleList<char>* third = new_Node('E');

   // link first node with the second node
   head->next = second;
   second->prev = head;

   // link second node with the third node
   second->next = third;
   third->prev = second;

   std::cout << "Initial Traversal" << std::endl;
   DllTraverse(head);

   // Insert 'G' at the head
   std::cout << "Inserting 'G' at Head" << std::endl;
   DllInsertHead('G', head);
   DllTraverse(head);

   // Insert 'G' at the end
   std::cout << "Inserting 'G' at End" << std::endl;
   DllInsertEnd('G', head);
   DllTraverse(head);

   // Insert 'B' inside the list after 'P' (second node)
   std::cout << "General Insertion ('B' after 'P')" << std::endl;
   DllgeneralInsert('B', second);
   DllTraverse(head);

   // Delete a specific node ('J')
   std::cout << "Deleting node 'J'" << std::endl;
   DllDelete('J', head);
   DllTraverse(head);

   // Clean up memory by deleting the entire list
   std::cout << "Deleting Entire List" << std::endl;
   DllDeleteList(head);
```

```
    if (head == nullptr) {
        std::cout << "List successfully deleted and head is null" <<
std::endl;
    }

    return 0;
}
```

OUTPUT

```
Initial Traversal
Forward Traversal:
C P E
Reverse Traversal:
E P C

Inserting 'G' at Head
Forward Traversal:
G C P E
Reverse Traversal:
E P C G

Inserting 'G' at End
Forward Traversal:
G C P E G
Reverse Traversal:
G E P C G
```

```
General Insertion ('B' after 'P')
Forward Traversal:
G C P B E G
Reverse Traversal:
G E B P C G

Deleting node 'J'
Forward Traversal:
G C P B E G
Reverse Traversal:
G E B P C G

Deleting Entire List
List successfully deleted and head is null
```

ANALYSIS

In this program, it creates a doubly linked list that stores one character in each node and connects each node using both next and prev pointers. The program can traverse the list in both forward and reverse directions, making it easier to move through the nodes. It also performs different operations such as inserting a node at the head, at the end, and after a specific node, as well as deleting a node and deleting the entire list. The output changes after every operation, showing how the doubly linked list is updated until all nodes are removed and the head becomes nullptr.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO B: Solve given problems utilizing linked lists in C++

Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing

Problem Description:

In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple Players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.
We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

SOURCE CODE

```
//supplementary.h
#ifndef Circular_LL_H
#define Circular_LL_H

#include <iostream>

template <typename T>
class CircularList {
public:
   T data;
   CircularList<T>* next = nullptr;
};

//creating a node
template <typename T>
CircularList<T>* new_Node(T newData) {

   //dynamically allocate memory for a new node
   CircularList<T>* newNode = new CircularList<T>;

   //store the data in the node
   newNode->data = newData;

   //since this is a new node, it points to itself
   newNode->next = newNode;

   return newNode;
```

```
}

//Traversal
template <typename T>
void CllTraverse(CircularList<T>* currentHead) {

    if (currentHead == nullptr) {
        std::cout << "Playlist is Empty." << std::endl;
        return;
    }

    CircularList<T>* currentNode = currentHead;

    do {
        std::cout << currentNode->data << " -> ";
        currentNode = currentNode->next;
    } while (currentNode != currentHead);
    std::cout << "Loop back to:" << currentNode->data;
    std::cout << std::endl << std::endl;
}

//Insertion at Head
template <typename T>
void CllInsertHead(T newData, CircularList<T>*& currentHead) {

    CircularList<T>* newNode = new_Node(newData);

    if (currentHead == nullptr) {
        currentHead = newNode;
        return;
    }

    CircularList<T>* tail = currentHead;

    while (tail->next != currentHead) {
        tail = tail->next;
    }

    newNode->next = currentHead;
    tail->next = newNode;
    currentHead = newNode;
}

//Insertion at End
template <typename T>
void CllInsertEnd(T newData, CircularList<T>*& currentHead) {

    CircularList<T>* newNode = new_Node(newData);

    if (currentHead == nullptr) {
        currentHead = newNode;
        return;
    }
```

```
    CircularList<T>* tail = currentHead;

    while (tail->next != currentHead) {
        tail = tail->next;
    }

    tail->next = newNode;
    newNode->next = currentHead;
}

// General Insertion
template <typename T>
void CllgeneralInsert(T newData, CircularList<T>* prevNode) {

    if (prevNode == nullptr) {
        std::cout << "Previous node cannot be null." << std::endl;
        return;
    }

    CircularList<T>* newNode = new_Node(newData);

    newNode->next = prevNode->next;
    prevNode->next = newNode;
}

//Deletion
template <typename T>
void CllDelete(T findData, CircularList<T>*& currentHead) {

    if (currentHead == nullptr)
        return;

    CircularList<T>* currNode = currentHead;
    CircularList<T>* prevNode = nullptr;

    do {

        if (currNode->data == findData) {

            // Delete head node
            if (prevNode == nullptr) {

                CircularList<T>* tail = currentHead;

                while (tail->next != currentHead) {
                    tail = tail->next;
                }

                // Only one node
                if (tail == currentHead) {
                    delete currentHead;
                    currentHead = nullptr;
```

```
                    return;
                }

                currentHead = currentHead->next;
                tail->next = currentHead;

                delete currNode;
                return;
            }

            // Delete middle or last node
            prevNode->next = currNode->next;
            delete currNode;
            return;
        }

        prevNode = currNode;
        currNode = currNode->next;

    } while (currNode != currentHead);

    std::cout << "Song not found." << std::endl;
}

//Delete Entire List
template <typename T>
void CllDeleteList(CircularList<T>*& currentHead) {

    if (currentHead == nullptr)
        return;

    CircularList<T>* currNode = currentHead->next;

    while (currNode != currentHead) {

        CircularList<T>* temp = currNode;
        currNode = currNode->next;

        delete temp;
    }

    delete currentHead;
    currentHead = nullptr;
}

#endif
```

```
//supplementary.cpp
#include <iostream>
#include "CPE010_HOA3_ILOB_OBIAS.h"

int main() {
```

```
    //music playlist
    CircularList<std::string>* head = new_Node<std::string>("Song1"); //head
node
    CircularList<std::string>* second = new_Node<std::string>("Song2");
//second node
    CircularList<std::string>* third = new_Node<std::string>("Song3");
//third node

    head->next = second; //first node pointing second node
    second->next = third; //second node pointing third node
    third->next = head; //third node pointing back to the head

    std::cout << "Initial Playlist" << std::endl;
    CllTraverse(head);

    //insert song at the head
    std::cout << "Inserting 'Song4' at Head" << std::endl;
    CllInsertHead<std::string>("Song4", head);
    CllTraverse(head);

    //insert song at the end
    std::cout << "Inserting 'Song5' at End" << std::endl;
    CllInsertEnd<std::string>("Song5", head);
    CllTraverse(head);

    //insert song after Photograph (second node)
    std::cout << "General Insertion ('Song6' after 'Song2')" << std::endl;
    CllgeneralInsert<std::string>("Song6", second);
    CllTraverse(head);

    //delete a song
    std::cout << "Deleting Song 'Song3'" << std::endl;
    CllDelete<std::string>("Song3", head);
    CllTraverse(head);

    //delete the entire playlist
    std::cout << "Deleting Entire Playlist" << std::endl;
    CllDeleteList(head);

    if (head == nullptr) {
        std::cout << "Playlist successfully deleted and head is null." <<
std::endl;
    }

    return 0;
}
```

OUTPUT

```
Initial Playlist
Song1 -> Song2 -> Song3 -> Loop back to:Song1

Inserting 'Song4' at Head
Song4 -> Song1 -> Song2 -> Song3 -> Loop back to:Song4

Inserting 'Song5' at End
Song4 -> Song1 -> Song2 -> Song3 -> Song5 -> Loop back to:Song4

General Insertion ('Song6' after 'Song2')
Song4 -> Song1 -> Song2 -> Song6 -> Song3 -> Song5 -> Loop back to:Song4

Deleting Song 'Song3'
Song4 -> Song1 -> Song2 -> Song6 -> Song5 -> Loop back to:Song4

Deleting Entire Playlist
Playlist successfully deleted and head is null.
```

CONCLUSION

In this lesson, I learned how to create and use a circular linked list by connecting the last node back to the head. I also learned how to perform different operations such as inserting, deleting, and traversing the list while making sure the links between the nodes remain correct. Overall, this activity helped me better understand how circular linked lists work and how each function updates the list after every operation.

Table 3-4. Code and Conclusion for Supplementary

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.